Lesson 11: Wave After Wave

Unity

VIDEO GAME CREATION

This week: clearing a wave of enemies no longer means the game is over - it means the NEXT wave begins. Clear all 3, and that's when you really win.

Open your own [YourName]Game project from Lesson 10 - don't start a new one.

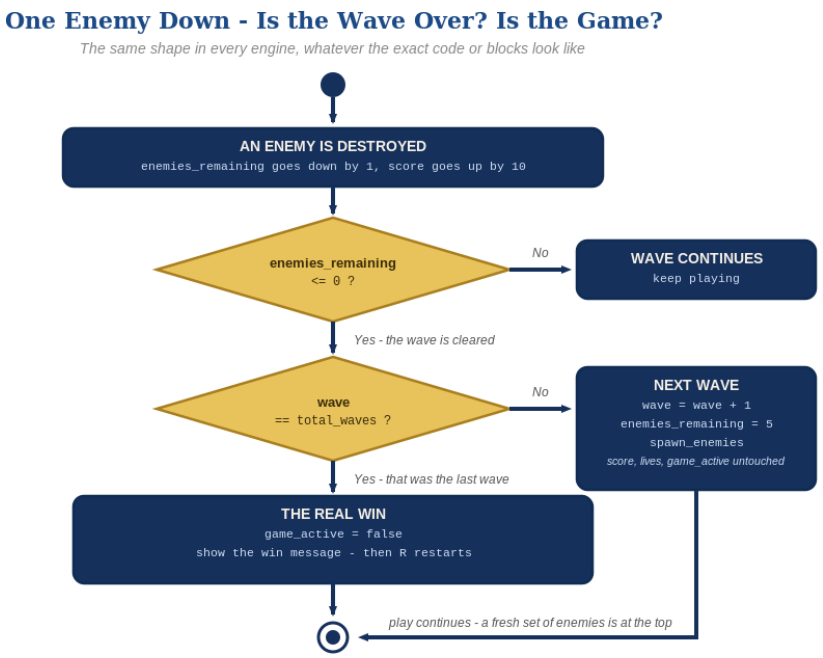
Tip: no new assets this week - waves reuse the spawn routine you already built in Lesson 10. This completes the Level 2 Complete milestone!

Setup

1. Open your [YourName]Game project from Lesson 10.
2. GameObject > UI > Legacy > Text, rename it WaveText, and position it near ScoreText and LivesText.

Level 1 - Minimal (MVP)

Clearing a wave's enemies spawns a fresh wave, up to 3 total. Clearing wave 3 triggers the real win. The current wave number is shown on screen.



spawn_enemies is the ONE routine that creates a wave: launch, restart and every new wave all call it. The enemy-creating code still exists only once.

VIDEO GAME CREATION

Clearing a wave asks a second question. If it was not the last wave, the game spawns a fresh set of enemies and carries on. The same shape in every engine.

1. In HUD.cs, add a public Text waveText field and a line in Update() that sets it from GameManager.wave and GameManager.totalWaves (code below). Then select the HUD GameObject and drag WaveText onto its new Wave Text slot in the Inspector.
2. In GameManager.cs, add public static int wave = 1, public static int totalWaves = 3 and public static bool nextWaveRequested = false (code below).
3. In RegisterKill(), revise the win check into an if/else: if this was the last wave, win as before. Otherwise advance wave, reset enemiesRemaining and raise the nextWaveRequested flag. Why a flag? RegisterKill() is static, so it belongs to the GameManager class, not to the GameManager object in your scene, and it cannot reach the Spawner slot you filled in last week. The flag is a note-to-self: GameManager's own Update() (which belongs to the object, and CAN use the Spawner slot) sees the note and spawns the wave.
4. In ResetGame(), add wave = 1 and nextWaveRequested = false to the resets.
5. In GameManager's Update(), add a check for nextWaveRequested: clear the flag, then call spawner.SpawnEnemies() (code below). EnemySpawner.cs does not change at all this week.
6. Press Play and play through. Clearing wave 1 or 2 should bring a fresh set of enemies instead of ending the game. Clearing wave 3 should show the win message.

Highlighted lines are new this week; everything else is code you already have.

```
// HUD.cs (complete):
using UnityEngine;
using UnityEngine.UI;
```

```
public class HUD : MonoBehaviour {
    public Text scoreText;
    public Text livesText;
    public Text waveText;

    void Update() {
        scoreText.text = "Score: " + GameManager.score;
        livesText.text = "Lives: " + GameManager.lives;
        waveText.text = "Wave: " + GameManager.wave + " of " +
GameManager.totalWaves;
    }
}

// GameManager.cs (the new and revised members):
public static int wave = 1;
public static int totalWaves = 3;
public static bool nextWaveRequested = false;

public static void RegisterKill() {
    enemiesRemaining--;
    score += 10;
    if (enemiesRemaining <= 0) {
        if (wave >= totalWaves) {
            hasWon = true;
            gameActive = false;
        } else {
            wave++;
            enemiesRemaining = 5;
            nextWaveRequested = true;
        }
    }
}

public void ResetGame() {
    spawner.ClearEnemies();
    score = 0;
    lives = 3;
    wave = 1;
    enemiesRemaining = 5;
    hasWon = false;
    hasLost = false;
    nextWaveRequested = false;
    gameActive = true;
    spawner.SpawnEnemies();
}

void Update() {
    if (!gameActive && Keyboard.current.rKey.wasPressedThisFrame) {
        ResetGame();
    }
    if (nextWaveRequested) {
        nextWaveRequested = false;
        spawner.SpawnEnemies();
    }
}
```

```
// EnemySpawner.cs: unchanged.
```

Before You Test: Check Your Game State

Remember Lesson 10: a restart must put ALL game state back. This week you added new state, so check it too:

1. wave and nextWaveRequested: confirm ResetGame() sets wave = 1 and nextWaveRequested = false. Static fields keep their values across restarts until you set them.
2. WaveText: HUD.cs updates it every frame, so it resets itself. Anything extra you added for waves (a message, a timer, a sound) needs its own reset inside ResetGame().

Level 2 - Complete (Alpha/Beta)

Wave transitions are seamless and correct every time - no double-spawns, no skipped waves, exactly 3 waves before the real win, and restarting always returns to wave 1. Nothing in the code needs to change for Level 2; it is about testing what you built.

- Play a full 3-wave game: confirm waves 1 and 2 each spawn a fresh set of enemies with no win message, and only wave 3 shows the win message.
- Let all 3 lives run out partway through wave 2 and confirm the lose message still appears, not a wave transition.
- Press R after a win or a loss and confirm wave is back to 1, along with score and lives, with exactly five enemies on screen.

Level 3 - Refined (Ready to Publish) - pick any that interest you

- The Stalemate is back - and now it can freeze the game at any of the three waves. If you fixed it in Lesson 10, check that your fix still works with waves. If you didn't, this may be the week it gets annoying enough to deal with. Find the cause and fix it in the way you, the Designer, think best. (Test it: let some enemies escape, destroy the rest, in wave 1, wave 2 and wave 3.)
- Visual: briefly flash a "Wave 2!" style message with its own GUI.Label and a short timer when a new wave spawns, separate from the win and lose labels.
- Structure: notice SpawnEnemies() is now called from three places (EnemySpawner's Start(), ResetGame() and the wave check in Update()), but the enemy-creating code exists only once.

If Something's Wrong

- Wave 1 immediately shows the win message: confirm the if/else checks wave >= totalWaves, and that the else is paired with that if.
- New waves never spawn: confirm GameManager's Update() has the nextWaveRequested check, and that RegisterKill()'s else branch sets nextWaveRequested = true.
- NullReferenceException on waveText: the Wave Text slot on the HUD GameObject is empty. Drag WaveText onto it.
- Restarting leaves wave at whatever it was: confirm ResetGame() sets wave = 1.

- A wave never ends after you let some enemies escape - no next wave, no win, no Game Over: that is the Stalemate from Lesson 10 (Level 3), not a typo in your code. Escaped enemies cost a life but never count toward clearing the wave.

Want More?

Student Resource List > Unity section, or learn.unity.com